Architecture

src/ — 2,000+ files · ~512K lines · 43 tools · 39 services

01 Folder Structure

src/ — 2,000+ files · ~512K lines of TypeScript

```
src/  2,000+ files · ~512K lines
├── entrypoints/  CLI bootstrap & entry points
│   ├── cli.tsx  Fast-path bootstrap (39KB)
│   ├── init.ts  Initialization sequence
│   ├── mcp.ts  MCP server mode entry
│   └── sdk/  Agent SDK type definitions
├── commands/  101 dirs · 207 files · CLI command handlers
│   ├── commit.ts  Git commit command
│   ├── config/  Settings management
│   ├── memory/  Memory CRUD commands
│   └── ... (100+ more)  doctor, upgrade, login, logout, etc.
├── tools/  43 tools · 184 files
│   ├── BashTool/  Shell command execution
│   ├── FileReadTool/  Filesystem reading
│   ├── FileEditTool/  Precise string replacement
│   ├── AgentTool/  Subagent spawning & orchestration
│   ├── MCPTool/  Dynamic MCP tool wrapper
│   ├── WebFetchTool/  HTTP fetch + HTML→markdown
│   └── ... (37+ more)  Blob, Grep, Write, TodoWrite, etc.
├── services/  39 services · 130 files
│   ├── api/  Claude API client (Anthropic SDK)
│   ├── mcp/  MCP protocol implementation (24 files)
│   ├── tools/  StreamingToolExecutor, registry
│   ├── compact/  Context compaction service
│   ├── analytics/  Telemetry & event tracking
│   ├── oauth/  Authentication flows
│   └── plugins/  Community plugin system
├── components/  144 files · Ink terminal UI components
├── hooks/  95 React hooks (useStream, useTools…)
├── ink/  Custom terminal renderer
│   ├── reconciler.ts  React reconciler for TTY
│   ├── layout/  Yoga flexbox engine bindings
│   └── termio/  ANSI escape sequence parser
└── utils/  564 files · Helper functions
    ├── permissions/  Permission system (24 files)
    ├── bash/  Bash AST parser (safety checks)
    ├── model/  Model selection & routing
    ├── settings/  Settings read/write layer
    ├── computerUse/  Computer control utilities
    └── git.ts  Git porcelain operations
├── state/  Centralized application state
├── bridge/  Remote control protocol
├── cli/  Transport layer (WS/SSE)
├── memdir/  Memory persistence layer
├── tasks/  Background task system
├── coordinator/  Multi-agent orchestration
├── assistant/  KAIROS persistent mode
├── voice/  Voice input pipeline
└── schemas/  JSON validation schemas
```

02 Boot Sequence

What happens when you type `claude`

01 · cli.tsx loads
Fast-path check — `--version` exits immediately with zero imports. Keeps startup under 50ms for simple flags.

02 · Feature flags evaluated
`bun:bundle` eliminates dead code paths at build time — feature-gated tools are tree-shaken away.

03 · main.tsx initializes
Commander.js parses argv. Subcommand routing dispatches to the matching command handler.

04 · Config loaded
Reads `settings.json`, all `CLAUDE.md` files in the project tree, and `.mcp.json` via dirwv.

05 · Auth checked
OAuth token from `~/.claude/auth.json`, or `ANTHROPIC_API_KEY` env var validated before any API call.

06 · GrowthBook initialized
Remote feature flags fetched asynchronously. Controls gradual rollouts of experimental features without deploys.

REPL initialization

07 · Tools assembled
43 built-in tools loaded from registry. MCP tool schemas merged in dynamically after server connection.

08 · MCP servers connected
stdio, SSE, and WebSocket transports negotiated in parallel. Capability exchange completes before loop starts.

09 · System prompt built
Assembled from 10+ component sources: role definition, tool schemas, CLAUDE.md, memory files, git context, OS info.

10 · REPL launched
Ink renders the terminal UI using a custom React reconciler backed by Yoga flexbox. Cursor positioned, input armed.

11 · Query loop begins
Event loop enters idle state, waiting for user input. Background tasks (MCP heartbeats, analytics flush) run on timers.

03 Query Loop — The Core Cycle

Input → API → Tools → Output

User types message
keyboard input or piped stdin

↓

`createUserMessage()`
wraps text in Anthropic message format

↓

Append to conversation history
in-memory array of messages

↓

Build system prompt
CLAUDE.md + tools + context + memory files

↓

Stream to Claude API
Anthropic SDK · server-sent events

↓

Parse response tokens as they arrive
renders incrementally to terminal

↓

if tool_use blocks found in response

↓

`findToolByName()`
look up in combined tool registry

↓

`canUseTool()`
check permissions: ask / allow / deny

↓

StreamingToolExecutor
run tools — concurrent-safe, parallel-capable

↓

Collect tool results
assemble tool_result message blocks

loop back to API with results

↓

Display final response to user
markdown rendered in terminal

↓

Post-sampling hooks
auto-compact · memory extract · dream mode

↓

Wait for next input
cycle restarts

Key implementation details

Streaming by default
Tokens render to the terminal as they arrive from the API. Users see output immediately — no waiting for the full response.

Parallel tool execution
When Claude requests multiple tools simultaneously, `StreamingToolExecutor` runs them concurrently. Results are collected and sent back together.

Agentic loop
Tool use → result → next API call cycles automatically. Loop terminates only when Claude emits a response with no tool_use blocks, or a stop condition triggers.

React rendering
Terminal UI re-renders via Ink's reconciler on every state change. Yoga computes flexbox layout; ANSI sequences are diffed to minimise redraws.

Interrupt handling
`Ctrl+C` during a tool call aborts the executor and sends a cancellation signal. The conversation history is preserved; a new query can continue.

Post-sampling hooks
After each assistant turn: context compaction threshold checked, memory extraction runs, and optional 'dream' consolidation pass fires if enabled.

04 Tool System — 43 Built-in Tools

File Operations & Execution

FILE OPERATIONS
`Read` `Edit` `Write` `Glob` `Grep` `NotebookEdit`

EXECUTION
`Bash` `PowerShell`

SEARCH & FETCH
`WebFetch` `WebSearch` `ToolSearch`

Agents, Planning & MCP

AGENTS
`Agent` `SendMessage` `TaskCreate` `TaskGet` `TaskList` `TaskUpdate` `TaskStop` `TaskOutput`

PLANNING & WORKTREES
`EnterPlanMode` `ExitPlanMode` `EnterWorktree` `ExitWorktree`

MCP
`MCPTool` `ListMcpResources` `ReadMcpResource`

System & Utility

SYSTEM
`AskUserQuestion` `TodoWrite` `Skill` `RemoteTrigger` `CronCreate` `CronDelete` `CronList` `Config`

Feature-gated / Experimental

GROWTHBOOK-CONTROLLED
`Sleep` `Brief` `WebBrowser` `TerminalCapture` `Monitor` `Workflow` `CtxInspect` `Snip` `OverflowTest` `VerifyPlanExecution` `ListPeers`

These tools are compiled into the binary but only surface in the tool registry when the corresponding GrowthBook feature flag is enabled for the account.

05 Permission System

3-layer permission model

1 · Tool Registry Filter
`filterToolsByDenyRules()` removes denied tools before Claude's context is built. Claude never sees — and cannot call — tools blocked at this layer. Configured via `settings.json` deny list.

2 · Per-call Permission Check
`canUseTool()` is called each time Claude attempts a tool call. Checks the allow/deny rule set against the specific invocation — tool name, arguments, and working directory pattern.

3 · Interactive User Prompt
If no rule matches a tool call, execution halts and the user is asked. Choices: `allow once` · `allow always` · `deny`. "Allow always" writes a new rule to settings.

- **Plan mode** — requires approval before execution
- **Auto mode** — AI decides, no prompting
- **Default** — prompt for risky tools only

Bash safety — AST-level analysis

The Bash tool includes a full shell AST parser in `utils/bash/`. Before executing any shell command, it parses the AST to detect dangerous patterns.

FLAGGED PATTERNS
`rm -rf /` `fork bombs` `curl | bash` `sudo escalation` `tty injection` `history manipulation`

Bash AST analysis runs before `canUseTool()`. If a pattern is flagged, the call is rejected immediately regardless of permission rules.

SETTINGS.JSON RULE FORMAT

```
{
  "permissions": {
    "allow": ["Bash(git *)"],
    "deny":  ["Bash(rm *)", "Write"]
  }
}
```

06 Context Management

Token budget — 200K context window (Claude 3.5 / 3.7 / Sonnet)

TYPICAL TOKEN DISTRIBUTION

Sys	Mem	Conversation history	Tool results	Available

AFTER COMPACT TRIGGERS (~80% FULL)

Sys	Mem	Summary	Recent	~40% freed